We now have a Dockerfile to create a production version of our image. There are further enhancements that we might want to make, but it's a good start. It allows us to deploy a production version of our app and see it running. Note that we haven't built the image yet: we'll get to that shortly.

Sharing Images

Up until now, during development, we've been building our custom images on our local machine. Initially, we did this with this command:

```
$ docker build [OPTIONS] .
```

We quickly progressed to using Compose to build images for us—for example:

```
$ docker-compose build web
```

However, we have a problem. We're going to need to run those same images we've been building locally on *different machines* as our app progresses through our build pipeline environments (such as testing, integration, staging, and production). How will we get our images onto the different machines?

One possibility is to rebuild the image on each machine that needs it, but this is wasteful and time-consuming. Images are more than just a convenient way to package up our code to be run in isolated containers. Since they contain everything needed to run the software, they are the perfect unit for sharing.

Docker has a built-in mechanism for distributing images. In fact, we've already seen it in action. Think back, for example, to one of our first Docker commands:

```
$ docker run ruby:2.6 ruby -e "puts :hello"
```

When we tried to run a container based on the ruby:2.6 image, Docker detected that we didn't have that image locally, and proceeded to download it.

```
Unable to find image 'ruby:2.6' locally
2.6: Pulling from library/ruby
cd8eada9c7bb: Pull complete
c2677faec825: Pull complete
fcce419a96b1: Pull complete
045b51e26e75: Pull complete
3b969ad6f147: Pull complete
f2db762ad32e: Pull complete
708e57760f1b: Pull complete
06478b05a41b: Pull complete
Digest: sha256:ad724f6982b4a7c2d2a8a4ecb67267a1961a518029244ed943e2d448d6fb7
994
Status: Downloaded newer image for ruby:2.6
hello
```